E4 Educator v2.0

Tier 3 · Project 2 of 5

P02

Data Logger

Walark Electronics · Fundrum Engineering · May 2026

Project overview

The Data Logger is a precision multi-sensor recording instrument. It introduces two capabilities not present in P01: real-time clock (DS3231 RTC) for proper timestamped entries, and daily log file rotation so each day produces a separate, named CSV file. The LittleFS web dashboard includes a Chart.js history graph that plots the last 60 readings. Data can be downloaded directly from the browser.

New skills beyond P01:

DS3231 RTC (I2C) · Proper ISO-8601 timestamps · Daily log rotation by date · Chart.js history graph in web dashboard · /api/log CSV download endpoint · BMP280 barometric pressure · Pressure trend arrow (rising/stable/falling) · Multi-sensor TFT layout across three pages

1 Project specification

1.1 Feature specification

ID	Feature	Detail
F01	AHT20 sensor	Temperature + humidity read on configurable interval (default 60s)
F02	DS18B20 probe	External probe temperature, non-blocking conversion, logged alongside AHT20
F03	BMP280 pressure	Barometric pressure hPa, trend arrow (↑ rising / → stable / ↓ falling) over 30-min window
F04	LDR light	Light level 0-100%, 8-sample averaged ADC1 read
F05	DS3231 RTC	Real-time clock I2C 0x68. ISO-8601 timestamp on every log entry: YYYY-MM-DD HH:MM:SS
F06	Daily log rotation	New CSV file per day: /LOG_YYYYMMDD.CSV. Midnight rotation handled automatically.
F07	SD card logging	CSV: timestamp, tempAHT, humidity, pressure, tempProbe, lightPct. Append each reading.
F08	TFT dashboard	Three pages: Live readings, Statistics + pressure trend, Log file browser.
F09	Touch controls	Footer: REFRESH · PAGE · SETTINGS. NEXT/ENT physical buttons also functional.
F10	Web history graph	Chart.js line graph on index.html shows last 60 readings. /api/history returns JSON array.
F11	CSV download	/api/log?file=LOG_YYYYMMDD.CSV streams the file for download. /api/files lists all CSV files.
F12	WiFi + MQTT	Sensor data published on each read. e4/sensor/pressure added. e4/firmware retained.
F13	NVS settings	logInterval, tempAlert, humidAlert, pressAlert, brightness via Settings.h
F14	OTA updates	ElegantOTA at /update. TFT progress bar. Log files preserved across firmware updates.

1.2 New hardware — DS3231 RTC and BMP280

Component	I2C address	Connection
DS3231 RTC module	0x68	SDA/SCL on I2C expansion header (I1C or I1C2). Backup coin cell CR2032 required for time retention on power loss.
BMP280 pressure	0x76 or 0x77	SDA/SCL on second I2C expansion header (AHT20-BMP280 header). SDO pin LOW = 0x76.

Run I2C scanner first

Before writing any code, connect both new modules and run the I2C scanner from T04. Confirm you see 0x38 (AHT20), 0x68 (DS3231), and 0x76 or 0x77 (BMP280) on the bus. If addresses conflict, the BMP280 SDO pin can be pulled HIGH to change its address to 0x77.

2 DS3231 real-time clock

The DS3231 is a precision I2C RTC with a built-in temperature-compensated crystal oscillator (TCXO). It keeps accurate time to within ±2 minutes per year and retains time during power loss via a backup CR2032 coin cell. It is the standard RTC choice for embedded data logging.

2.1 Library setup

```
; platformio.ini lib_deps additions:
  adafruit/RTClib @ 2.1.1
  adafruit/Adafruit BMP280 Library @ 2.6.6
```

2.2 RTC initialisation and time setting

```
#include <RTClib.h>

RTC_DS3231 rtc;

bool initRTC() {
  if (!rtc.begin(&Wire)) {
    Serial.println("DS3231 not found."); return false;
  }

  if (rtc.lostPower()) {
    // RTC lost power — set to compile time as starting point
    // In production: sync from NTP once WiFi connects
    rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
    Serial.println("RTC set from compile time.");
  }

  DateTime now = rtc.now();
  Serial.printf("RTC time: %04d-%02d-%02d %02d:%02d:%02d\n",
    now.year(), now.month(),  now.day(),
    now.hour(), now.minute(), now.second());
  return true;
}

// Sync RTC from NTP once WiFi connects (call once after WiFi OK)
void syncRTCFromNTP() {
  configTime(46800, 0, "pool.ntp.org");  // UTC+13 for NZ
  struct tm timeInfo;
  if (getLocalTime(&timeInfo, 5000)) {
    rtc.adjust(DateTime(
      timeInfo.tm_year + 1900,
      timeInfo.tm_mon  + 1,
      timeInfo.tm_mday,
      timeInfo.tm_hour,
      timeInfo.tm_min,
      timeInfo.tm_sec));
    Serial.println("RTC synced from NTP.");
  } else {
    Serial.println("NTP sync failed — using existing RTC time.");
```

```
  }
}

// Get ISO-8601 timestamp string
String getTimestamp() {
  DateTime now = rtc.now();
  char buf[20];
  snprintf(buf, sizeof(buf), "%04d-%02d-%02d %02d:%02d:%02d",
           now.year(), now.month(),  now.day(),
           now.hour(), now.minute(), now.second());
  return String(buf);
}

// Get log filename for today: /LOG_YYYYMMDD.CSV
String todayLogFile() {
  DateTime now = rtc.now();
  char buf[24];
  snprintf(buf, sizeof(buf), "/LOG_%04d%02d%02d.CSV",
           now.year(), now.month(), now.day());
  return String(buf);
}
```

Key point

rtc.lostPower() returns true when the backup battery is flat or missing and the RTC has lost its time. Setting from compile time gives a reasonable starting timestamp. NTP sync after WiFi connects then corrects it to real time. After the first NTP sync, the RTC keeps accurate time independently of WiFi.

3 BMP280 barometric pressure

The BMP280 measures barometric pressure (300-1100 hPa) and temperature. Barometric pressure is a useful weather indicator: rising pressure typically indicates improving weather, falling pressure indicates deteriorating conditions. The Data Logger tracks the 30-minute pressure change to show a trend arrow on the dashboard.

3.1 BMP280 reading

```
#include <Adafruit_BMP280.h>

Adafruit_BMP280 bmp(&Wire);

float pressureHPa  = 1013.25f;
float bmpTempC     = 0.0f;

// Pressure history for trend detection
#define PRESSURE_HISTORY 30  // 30 samples at 1 per minute = 30 min
float pressHistory[PRESSURE_HISTORY] = {};
int   pressHistIdx = 0;
bool  pressHistFull = false;

bool initBMP280() {
  if (!bmp.begin(0x76)) {
    // Try alternate address
    if (!bmp.begin(0x77)) {
      Serial.println("BMP280 not found."); return false;
    }
  }
  bmp.setSampling(Adafruit_BMP280::MODE_NORMAL,
                  Adafruit_BMP280::SAMPLING_X2,
                  Adafruit_BMP280::SAMPLING_X16,
                  Adafruit_BMP280::FILTER_X16,
                  Adafruit_BMP280::STANDBY_MS_500);
  return true;
}

void readBMP280() {
  pressureHPa = bmp.readPressure() / 100.0f;
  bmpTempC    = bmp.readTemperature();
  // Store in circular history buffer
  pressHistory[pressHistIdx] = pressureHPa;
  pressHistIdx = (pressHistIdx + 1) % PRESSURE_HISTORY;
  if (pressHistIdx == 0) pressHistFull = true;
}

// Returns: 1=rising, 0=stable, -1=falling
int pressureTrend() {
  if (!pressHistFull) return 0;  // Not enough data yet
  int oldest = pressHistIdx;
  float first = pressHistory[oldest];
  float last  = pressureHPa;
  float delta = last - first;
```

```
  if (delta >  1.0f) return  1;  // Rising > 1hPa in 30 min
  if (delta < -1.0f) return -1;  // Falling
  return 0;                       // Stable
}

const char* trendArrow() {
  switch (pressureTrend()) {
    case  1: return "\u2191 Rising";
    case -1: return "\u2193 Falling";
    default: return "\u2192 Stable";
  }
}
```

4 Daily log rotation with RTC timestamps

Daily log rotation creates a new CSV file at midnight each day, named with the date. This is far more useful than P01's size-based rotation for long-term data analysis — you can instantly locate the data from any specific date, and each file covers exactly 24 hours.

4.1 Logger module — daily rotation

```
// Logger.h additions for P02
#include <RTClib.h>
extern RTC_DS3231 rtc;

String currentLogFile = "";

// Call at startup and after midnight
void updateLogFile() {
  String todayFile = todayLogFile();  // from RTC module

  if (todayFile == currentLogFile) return;  // same day

  currentLogFile = todayFile;
  Serial.printf("Log file: %s\n", currentLogFile.c_str());

  if (!SD.exists(currentLogFile.c_str())) {
    File f = SD.open(currentLogFile.c_str(), FILE_WRITE);
    if (f) {
      f.println("timestamp,tempAHT,humidity,pressHPa,tempProbe,lightPct");
      f.close();
      Serial.println("New log file created.");
    }
  }
}

bool logReading() {
  if (!sdOk || !logEnabled) return false;

  updateLogFile();  // Handles midnight rotation transparently

  char line[80];
  snprintf(line, sizeof(line),
           "%s,%.2f,%.1f,%.2f,%.2f,%d",
           getTimestamp().c_str(),
           ahtTempC, ahtHumid,
           pressureHPa, ds18TempC,
           lightPct);

  File f = SD.open(currentLogFile.c_str(), FILE_APPEND);
  if (!f) return false;
  f.println(line);
  f.close();
  logRows++;

  Serial.printf("Logged [%s] T:%.1f H:%.1f P:%.1f\n",
```

```
                getTimestamp().c_str(),
                ahtTempC, ahtHumid, pressureHPa);
  return true;
}

// List all CSV log files on SD
String listLogFiles() {
  String result = "[";
  bool first = true;
  File root = SD.open("/");
  File f = root.openNextFile();
  while (f) {
    String name = f.name();
    if (name.startsWith("LOG_") && name.endsWith(".CSV")) {
      if (!first) result += ",";
      result += "{\"name\":\"" + name + "\",";
      result += "\"size\":" + String(f.size()) + "}";
      first = false;
    }
    f.close(); f = root.openNextFile();
  }
  root.close();
  return result + "]";
}
```

Key point

updateLogFile() is called at the start of every logReading(). It compares the RTC date to the current log filename and rotates silently at midnight. The loop running at 1-minute intervals will catch the midnight rotation within 60 seconds of midnight. No special midnight timer is needed.

5 Web dashboard with Chart.js history graph

The P02 web dashboard extends the P01 design with a Chart.js line graph showing the last 60 readings. The /api/history endpoint returns a JSON array that the chart updates automatically every 60 seconds.

5.1 /api/history endpoint

The history endpoint reads the last N lines from the current log file and returns them as a JSON array. This keeps the response size bounded regardless of how long the logger has been running.

```
// In Network.h — history endpoint
server.on("/api/history", HTTP_GET, [](AsyncWebServerRequest* req) {
  // Read last 60 log entries
  const int MAX_HISTORY = 60;
  File f = SD.open(currentLogFile.c_str());
  if (!f) { req->send(200, "application/json", "[]"); return; }

  // Collect all lines
  std::vector<String> lines;
  String line;
  while (f.available()) {
    line = f.readStringUntil('\n');
    line.trim();
    if (line.length() > 0 && !line.startsWith("timestamp"))
      lines.push_back(line);
  }
  f.close();

  // Take last MAX_HISTORY entries
  int start = max(0, (int)lines.size() - MAX_HISTORY);
  String json = "[";
  for (int i = start; i < (int)lines.size(); i++) {
    // Parse CSV line: timestamp,tempAHT,humidity,pressure,probe,light
    if (i > start) json += ",";
    // Split on commas and build JSON object
    int c1 = lines[i].indexOf(',');
    int c2 = lines[i].indexOf(',', c1+1);
    int c3 = lines[i].indexOf(',', c2+1);
    int c4 = lines[i].indexOf(',', c3+1);
    int c5 = lines[i].indexOf(',', c4+1);
    if (c1 > 0 && c2 > c1) {
      json += "{\"ts\":\"" + lines[i].substring(0, c1) + "\"";
      json += ",\"t\":" + lines[i].substring(c1+1, c2);
      json += ",\"h\":" + lines[i].substring(c2+1, c3);
      json += ",\"p\":" + lines[i].substring(c3+1, c4);
      json += "}";
    }
  }
  json += "]";
  req->send(200, "application/json", json);
});

// CSV download endpoint
```

```
server.on("/api/log", HTTP_GET, [](AsyncWebServerRequest* req) {
  if (!req->hasParam("file")) {
    req->send(400, "text/plain", "Missing file param"); return;
  }
  String filename = "/" + req->getParam("file")->value();
  if (!SD.exists(filename.c_str())) {
    req->send(404, "text/plain", "Not found"); return;
  }
  req->send(SD, filename.c_str(), "text/csv",
            true);  // true = download attachment
});

// List log files endpoint
server.on("/api/files", HTTP_GET, [](AsyncWebServerRequest* req) {
  req->send(200, "application/json", listLogFiles());
});
```

5.2 data/index.html with Chart.js

Create this file in your project's data/ folder. It includes the live sensor cards from P01 plus a Chart.js temperature/humidity dual-axis graph and a log file download section:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <title>E4 Data Logger</title>
  <script
src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js"></script>
  <style>
    body { font-family: Arial, sans-serif; background:#0d1117;
           color:#e6edf3; margin:0; padding:16px; }
    h2   { color:#2E86AB; margin:16px 0 8px; }
    .cards { display:flex; gap:12px; flex-wrap:wrap; }
    .card { background:#161b22; border-radius:10px; padding:16px;
            min-width:140px; }
    .lbl  { color:#8b949e; font-size:0.8em; }
    .val  { font-size:2em; font-weight:bold; margin:4px 0; }
    .chart-wrap { background:#161b22; border-radius:10px;
                  padding:16px; margin-top:16px; }
    .files a { color:#2E86AB; display:block; margin:4px 0; }
  </style>
</head>
<body>
  <h2>E4 Data Logger</h2>
  <div class="cards">
    <div class="card">
      <div class="lbl">AIR TEMP</div>
      <div class="val" id="temp" style="color:#2dc653">--</div>
    </div>
    <div class="card">
      <div class="lbl">HUMIDITY</div>
```

```
      <div class="val" id="humid" style="color:#2E86AB">--</div>
    </div>
    <div class="card">
      <div class="lbl">PRESSURE</div>
      <div class="val" id="press" style="color:#f4a261">--</div>
      <div id="trend" style="color:#8b949e;font-size:0.85em">→ Stable</div>
    </div>
    <div class="card">
      <div class="lbl">PROBE TEMP</div>
      <div class="val" id="probe" style="color:#79c0ff">--</div>
    </div>
  </div>

  <div class="chart-wrap">
    <h2>History — last 60 readings</h2>
    <canvas id="histChart" height="100"></canvas>
  </div>

  <div class="chart-wrap">
    <h2>Log files</h2>
    <div class="files" id="files">Loading...</div>
  </div>

  <script>
    const ctx = document.getElementById("histChart").getContext("2d");
    const chart = new Chart(ctx, {
      type: "line",
      data: { labels:[], datasets:[
        { label:"Temp °C", data:[], borderColor:"#2dc653",
          tension:0.3, pointRadius:2 },
        { label:"Humidity %", data:[], borderColor:"#2E86AB",
          tension:0.3, pointRadius:2, yAxisID:"y2" }
      ]},
      options: { responsive:true,
        scales: {
          y:  { position:"left",  ticks:{color:"#2dc653"} },
          y2: { position:"right", ticks:{color:"#2E86AB"},
grid:{drawOnChartArea:false} },
          x:  { ticks:{ color:"#8b949e", maxTicksLimit:10 } }
        },
        plugins:{ legend:{ labels:{ color:"#e6edf3" } } }
      }
    });

    async function refreshSensors() {
      const r = await fetch("/api/sensors");
      const d = await r.json();
      document.getElementById("temp").textContent
        = d.temp.toFixed(1) + " °C";
      document.getElementById("humid").textContent
        = d.humid.toFixed(1) + " %";
      document.getElementById("press").textContent
        = d.press.toFixed(1) + " hPa";
      document.getElementById("trend").textContent
```

```
        = d.trend;
      document.getElementById("probe").textContent
        = d.probe > -90 ? d.probe.toFixed(1)+" °C" : "--";
    }

    async function refreshHistory() {
      const r = await fetch("/api/history");
      const data = await r.json();
      chart.data.labels = data.map(d => d.ts.substring(11,16));
      chart.data.datasets[0].data = data.map(d => d.t);
      chart.data.datasets[1].data = data.map(d => d.h);
      chart.update();
    }

    async function refreshFiles() {
      const r = await fetch("/api/files");
      const files = await r.json();
      const div = document.getElementById("files");
      if (files.length === 0) { div.textContent = "No log files."; return; }
      div.innerHTML = files.map(f =>
        `<a href="/api/log?file=${f.name}" download>${f.name}
         (${Math.round(f.size/1024)} KB)</a>`
      ).join("");
    }

    refreshSensors(); refreshHistory(); refreshFiles();
    setInterval(refreshSensors, 10000);
    setInterval(refreshHistory, 60000);
    setInterval(refreshFiles,   60000);
  </script>
</body>
</html>
```

6 TFT dashboard — three pages

P02 extends the two-page P01 dashboard to three pages, adding the pressure trend and log file status as a dedicated second page. The static chrome includes all five sensor labels drawn once.

Page	Title	Content
PAGE_LIVE	Live readings	AHT20 temp · Humidity · DS18B20 probe · Light bar
PAGE_WEATHER	Weather	BMP280 pressure (large) · Trend arrow · Min/Max AHT20 · Reading count
PAGE_LOG	Log status	Today's log file name · Row count · File size · SD free space · Log on/off indicator · Last timestamp

6.1 Weather page — pressure display

```
void updateWeatherPage(float pressHPa, float bmpTemp,
                       float minT, float maxT, long reads) {
  spr.fillSprite(COL_BG);

  // Large pressure value
  spr.setTextColor(COL_ACCENT, COL_BG);
  spr.setTextSize(3);
  spr.setCursor(8, 8);
  spr.printf("%.1f hPa", pressHPa);

  // Trend arrow — big and prominent
  int trend = pressureTrend();
  uint16_t trendCol = (trend > 0) ? COL_OK :
                      (trend < 0) ? COL_ERR : COL_LABEL;
  spr.setTextColor(trendCol, COL_BG);
  spr.setTextSize(3);
  spr.setCursor(8, 42);
  spr.print(trendArrow());

  // 30-min change in hPa
  spr.setTextSize(1);
  spr.setTextColor(COL_LABEL, COL_BG);
  spr.setCursor(8, 78);
  spr.printf("30-min window  BMP T:%.1fC", bmpTemp);

  // Min/Max
  spr.setCursor(8, 96);
  spr.printf("Air T: lo %.1f  hi %.1f C", minT, maxT);

  // Reading count
  spr.setCursor(8, 110);
  spr.printf("Readings: %ld", reads);

  spr.pushSprite(0, 48);
}
```

7 main.cpp — P02 additions

P02 main.cpp follows the same coordinator structure as P01. The key additions are RTC initialisation, NTP sync after WiFi connects, BMP280 reading on each cycle, and three-page display routing.

```
// P02 additions to P01 main.cpp

#include "RTCmodule.h"    // NEW: RTC + BMP280 module

bool rtcOk  = false;
bool bmpOk  = false;
bool ntpSynced = false;

void setup() {
  // ... (same as P01 up to sensor init) ...

  // P02 new sensors
  rtcOk = initRTC();
  bmpOk = initBMP280();

  if (!rtcOk) Serial.println("RTC not found — timestamps use millis()");
  if (!bmpOk) Serial.println("BMP280 not found — pressure disabled");

  // ... rest of setup same as P01 ...
}

void loop() {
  unsigned long now = millis();
  wifiUpdate(now); mqttUpdate(now);
  ElegantOTA.loop(); sensorsUpdate(now);
  tonePlayer.update();

  // One-time NTP sync after WiFi connects
  if (wifiReady() && !webStarted) {
    setupWebServer(); webStarted = true;
    if (rtcOk && !ntpSynced) {
      syncRTCFromNTP(); ntpSynced = true;
    }
  }

  Button::Event nextEv = nextBtn.read();
  Button::Event entEv  = entBtn.read();
  uint16_t tx, ty;
  bool touched = tft.getTouch(&tx, &ty);

  handleButtons(nextEv, entEv);
  bool forceRead = false;
  handleTouch(tx, ty, touched, forceRead, now);

  bool shouldRead = forceRead ||
                    (now - lastSensorRead >= settings.s.logInterval);
  if (shouldRead) {
    lastSensorRead = now;
```

```
    readAllSensors();    // AHT20 + LDR
    if (bmpOk) readBMP280();
    checkAlerts();
    // Route to correct display page
    switch (currentPage) {
      case PAGE_LIVE:
        updateLivePage(ahtTempC, ahtHumid, ds18TempC, lightPct,
                       settings.s.tempAlert, settings.s.humidAlert);
        break;
      case PAGE_WEATHER:
        updateWeatherPage(pressureHPa, bmpTempC,
                          minTemp, maxTemp, readCount);
        break;
      case PAGE_LOG:
        updateLogPage(currentLogFile, logRows,
                      logFileSize(), logEnabled,
                      getTimestamp());
        break;
    }
    logReading();       // Timestamped, daily rotated
    publishSensors();   // Includes pressure
  }

  if (now - lastFooter >= 1000) {
    lastFooter = now;
    updateFooter(now, wifiReady(), mqtt.connected(), logEnabled);
  }

  if (!otaValid && now > 10000) {
    esp_ota_mark_app_valid_cancel_rollback();
    otaValid = true;
  }
}
```

8 Settings.h — P02 additions

P02 adds two new settings to the Settings struct: pressAlert for barometric pressure threshold and logInterval extended to support longer intervals (up to 60 minutes for low-power deployments).

```
// Settings.h additions for P02
// Add to Settings struct:
  float pressAlert   = 980.0;   // Alert if pressure falls below
  unsigned long logInterval = 60000;  // Default 60s (was 5s in P01)

// Add NVS key constants:
#define NVS_PRESS_AL  "pressAlert"   // 10 chars ✓

// Add to load():
  p.begin(NVS_NS_ALERT, true);
  s.pressAlert = p.getFloat(NVS_PRESS_AL, s.pressAlert);
  p.end();

// Add to save():
  p.begin(NVS_NS_ALERT, false);
  p.putFloat(NVS_PRESS_AL, s.pressAlert);
  p.end();

// Updated settings items for TFT settings page:
const SettingItem SETTINGS_ITEMS[] = {
  { "Brightness",    50, 255,  10 },
  { "Temp alert",    20,  40,   1 },
  { "Humid alert",   40,  90,   5 },
  { "Press alert",  960, 1020,  2 },
  { "Log interval",   1,  60,   1 },  // minutes
};
```

9 Commissioning checklist

☐	Step
☐	Run I2C scanner. Confirm 0x38 (AHT20), 0x68 (DS3231), 0x76 (BMP280) all visible.
☐	Insert CR2032 coin cell into DS3231 module before first power-on.
☐	Format microSD FAT32. Insert. Upload firmware then filesystem.
☐	On first boot: RTC time is set from compile time. Serial shows RTC time.
☐	Confirm Serial shows: AHT20 OK, DS18B20 found, DS3231 OK, BMP280 OK, SD OK.
☐	Check TFT page 1 shows all four sensor readings updating.
☐	NEXT to page 2: confirm BMP280 pressure displayed with trend arrow.
☐	NEXT to page 3: confirm log file name, row count, and file size visible.
☐	After WiFi connects: Serial confirms NTP sync. RTC time updates to local time.
☐	Check log file via web dashboard: open /api/files, confirm today's file listed.

☐	Click the log file download link in browser. Open CSV in Excel and verify timestamps.
☐	Wait 5+ readings then open browser. Confirm Chart.js history graph shows data.
☐	Check pressure trend: over 30 minutes the trend arrow should stabilise to ↑/↓/→.
☐	Confirm MQTT: e4/sensor/pressure topic updating in MQTT Explorer.
☐	Power cycle. Confirm RTC retains correct time (CR2032 present).
☐	At midnight (or set RTC ahead by hand and wait): confirm new log file created.

10 Extending the project

- Add a second Chart.js graph on index.html for pressure history to visualise weather patterns
- Add a /api/settings GET/POST endpoint so thresholds can be updated from the web browser without reflashing
- Add a Node-RED flow that generates a daily summary email with min/max temperature, humidity, and pressure from the previous day's log file
- Add a wind speed/direction sensor (SparkFun Weather Meter Kit on ADC1 + pulse counter) for a full weather station
- Calculate altitude from pressure using the barometric formula and display as an elevation badge on the weather page
- Add weekly log archiving: at the start of each week, zip the previous 7 daily files into a WEEK_YYYYWNN.zip on SD
- Add a Node-RED dashboard with a monthly temperature heat-map calendar view using the daily log files as source data

E4 Educator v2.0 · P02: Data Logger · Walark Electronics · Fundrum Engineering · May 2026